

SLDE-AFT: A Self-Learning Data Extraction and Automated Fine-Tuning System for Structured and Unstructured Data

Shashank Kumar Sadashiva¹, Ahmed Ebada¹

¹Hopn Ug, Germany

Abstract

Keywords

1 Introduction

The volume and structural heterogeneity of digital information have grown substantially across enterprise, scientific, and industrial domains [16]. Organizations routinely accumulate knowledge distributed across structurally dissimilar sources: relational databases, spreadsheets, and JSON repositories encode entity attributes with high fidelity, while technical reports, product specifications, and research literature encode complementary relational facts in natural language [1, 16]. The co-existence of these two source types creates a persistent fragmentation problem: structured pipelines apply schema-driven transformations that discard textual nuance, whereas unstructured extraction systems operate on free text without awareness of structured constraints [4, 25]. Although both modalities often describe the same real-world entities, the tools available to integrate and reconcile them have remained fundamentally separate, leaving a large proportion of organizationally valuable knowledge either inaccessible or duplicated without consolidation [4, 22].

Current information extraction and model adaptation pipelines address individual stages of the knowledge discovery process but do not form a coherent self-improving system [36]. Automated machine learning platforms such as Auto-Sklearn [6, 45] and Amazon SageMaker Autopilot [20] automate model selection and pipeline construction but treat data extraction as a fixed upstream preprocessing step

that lies outside the adaptive boundary. Retrieval-Augmented Generation systems [19, 54] improve factual grounding at inference time by accessing external corpora, yet the underlying extraction model is never updated from the retrieved or accumulated knowledge. Standard parameter-efficient fine-tuning methods, including Low-Rank Adaptation [29] and quantized variants [33], reduce the computational cost of model adaptation but depend on externally curated annotated datasets and require manually triggered discrete retraining cycles. To the best of our knowledge, no existing system connects the output of an extraction pipeline directly to a fine-tuning input without human curation at any intermediate stage.

The consequences of this fragmentation are concrete and cumulative. Open Information Extraction systems [4] demonstrated large-scale schema-free relation extraction, yet the paradigm they introduced has, to the best of our knowledge, remained predominantly unidirectional: extracted triples are typically stored, ranked, or consumed downstream rather than being used to retrain the extractor itself. Probabilistic knowledge representation frameworks such as Probabilistic Soft Logic [5] provide principled confidence models for accumulated facts, but these representations are constructed offline and carry no feedback channel to an active extraction model. Continual learning research [30] has established that sequential adaptation is achievable without catastrophic forgetting, yet existing continual learning systems operate over fixed supervised task streams rather than knowledge accumulated from heterogeneous documents. Self-improving language model systems, including SELF-REFINE [31], Reinforced Self-Training [36], and Self-Play Fine-Tuning [41], iterate on generation quality through self-generated feedback but maintain no structured, persistent knowledge representation that could systematically drive downstream adaptation. The result is that accumulated extraction output is rarely used to systematically improve the extractor itself, and fine-tuning typically depends on labels supplied externally or through separate annotation processes.

This paper introduces SLDE-AFT, a Self-Learning Dual-Source Knowledge Extraction and Automated Fine-Tuning system that addresses this gap by unifying five previously disconnected capabilities within a single seven-module architecture. SLDE-AFT simultaneously ingests structured inputs (CSV, JSON) and unstructured inputs (plain text) and maps both into a unified triple representation (*subject*, *predicate*, *object*, *confidence*, *source_id*, *provenance*).

Extracted triples are accumulated in a Probabilistic Knowledge Base (PKB) governed by a Conservative Noisy-Or aggregation formula with a mutual-exclusivity penalty, defined formally in Section 4.3

High-confidence triples ($C > 0.88$) are automatically converted into instruction-response training pairs following the self-supervised supervision paradigm established by SELF-INSTRUCT [24] and instruction-tuned models [10, 50]. These pairs trigger automated LoRA fine-tuning [29] without requiring manual label curation or explicit fine-tuning schedules. A Feedback Controller operating in Encouraging Mode—directing extraction toward underrepresented predicates—and Prohibitive Mode—suppressing saturated concepts—dynamically steers each extraction

iteration, reducing reliance on the human-in-the-loop signal that prior adaptive systems typically require [31, 36]. Every extracted triple is further linked to its source document via a provenance text window capturing the originating passage, and this provenance record actively filters synthetic training data quality rather than serving purely as an audit log [37, 49].

Experiments conducted across four dataset scales — 20, 30, 40, and 50 products — demonstrate measurable improvements through closed-loop iteration. On the 40-product configuration, SLDE-AFT achieves a precision of 0.9000 at Iteration 4 and the Probabilistic KB variant maintains Recall = 1.0000 across all iterations, with the strongest gains observed relative to all three baselines.

The contributions of this paper are as follows:

1. We propose SLDE-AFT, a unified dual-source closed-loop framework that, integrates structured and unstructured knowledge extraction within a common triple representation and connects extraction, probabilistic knowledge accumulation, synthetic supervision generation, and automated LoRA-based adaptation without requiring manual label curation during iterative operation [10, 24, 29].
2. We introduce an automated synthetic supervision mechanism that generates instruction-response training pairs directly from high-confidence knowledge base triples ($C > 0.88$), targeting coverage gaps between accumulated knowledge and the current extraction state, substantially reducing the annotation bottleneck and supporting iterative self-improvement without external supervision [10, 29, 33].
3. We develop a Conservative Noisy-Or aggregation formula extended with a mutual-exclusivity penalty for probabilistic confidence scoring in the knowledge base, providing a mathematically grounded uncertainty model for extraction-knowledge-base integration [5, 15].
4. We design an automated feedback controller that dynamically steers extraction across iterations through Encouraging Mode — directing extraction toward undercovered predicates — and Prohibitive Mode — suppressing saturated concepts — reducing reliance on the manual feedback signal required by prior adaptive systems. [31, 36, 50].
5. We incorporate provenance traceability into the fine-tuning loop by embedding provenance text windows and confidence scores as grounding evidence fields within each synthetic training example, ensuring source-traceability throughout iterative self-improvement rather than serving purely as an audit record [37, 49].

The remainder of this paper is organized as follows. Section 2 reviews related work across six paradigms: AutoML, information extraction, retrieval-augmented generation, parameter-efficient fine-tuning, self-improving learning systems, and knowledge representation. Section 3 presents the cross-paradigm research gap analysis. Section 4 describes the proposed seven-module SLDE-AFT architecture in full. Section 5 details the prototype implementation including dataset construction,

software stack, and experimental workflow. Section 6 defines the experimental setup including baselines, evaluation metrics, and gold standard construction. Section 7 reports results across all four dataset scales and all baseline comparisons. Section 8 presents the ablation analysis. Section 9 discusses key findings on precision improvement, recall preservation, and scalability behaviour. Section 10 addresses limitations and future work. Section 11 concludes the paper.

2 Related Work

The work presented in this paper intersects six established research paradigms: automated machine learning [1, 3, 6, 16], information extraction [4, 21, 22, 25], retrieval-augmented generation [17, 19], parameter-efficient fine-tuning [14, 28, 29, 33], feedback-based self-improvement [24, 31, 36, 41], and probabilistic knowledge representation [5, 15, 51]. Each paradigm addresses a partial subset of the capabilities that SLDE-AFT integrates within a single unified architecture.

2.1 Automated Machine Learning

Automated machine learning (AutoML) emerged to reduce human effort in constructing supervised learning pipelines by automating model selection, hyperparameter optimisation, and architecture search [1, 16]. Bayesian-optimisation-based frameworks such as Auto-Sklearn [45] and its successor Auto-Sklearn 2.0 [6] introduced meta-learning-guided warm starting and portfolio construction to reduce pipeline search time substantially. Ensemble-driven systems such as AutoGluon [18] extended automation to tabular prediction tasks through multi-layer stacking, while commercial platforms such as Amazon SageMaker Autopilot [20] enabled transparent, editable pipeline generation at deployment scale. Neural architecture search methods [3] further widened the automation boundary to model structure itself. Despite these advances, all AutoML frameworks treat data extraction as a fixed upstream preprocessing step: the adaptive boundary begins at feature engineering and ends at model deployment, leaving the connection between heterogeneous source ingestion, knowledge accumulation, and iterative model adaptation entirely outside the automation scope — a gap that the closed-loop design of SLDE-AFT directly addresses.

2.2 Information Extraction and Relation Extraction

Information extraction research has pursued two broad trajectories: schema-free open extraction and schema-guided structured extraction. Open Information Extraction systems [4, 25] demonstrated that relational triples could be extracted from large text corpora without predefining relation types, using lightweight linguistic processing and redundancy-based confidence estimation. Document-level relation extraction benchmarks such as DocRED [47] established evaluation standards for reasoning across sentence boundaries, while supervised relation classification datasets such as TACRED [48] became widely adopted benchmarks for sentence-level relation extraction. Generative extraction architectures reformulated triple extraction as sequence-to-structure generation: REBEL [21] reformulated end-to-end relation extraction as autoregressive triplet generation, GenIE [23] extended this to schema-constrained decoding at large entity-relation scale, and TANL [27] treated structured prediction as translation between augmented natural languages. Universal IE frameworks [22, 32] unified heterogeneous extraction tasks under a single architecture via structured extraction languages and multi-task instruction tuning. Joint extraction systems such as DyGIE++ [56] improved the integration of entity and relation extraction, while open-world attribute discovery systems such as OAMine [38] enabled the discovery of previously unseen attributes beyond predefined schemas. Zero-shot relation extraction approaches [46] demonstrated that question-template reformulation can facilitate extraction of previously unseen relation types without retraining. While these approaches improve extraction accuracy, coverage, schema flexibility, or generalization capability, they generally treat extraction as a terminal process and do not explicitly leverage accumulated extracted knowledge to drive iterative model adaptation through an automated feedback loop.

2.3 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) addresses the knowledge limitations of parametric language models by augmenting generation with dynamically retrieved external evidence, thereby reducing hallucination and improving factual grounding. The foundational RAG framework [19, 54] demonstrated that dense passage retrieval combined with sequence-to-sequence generation improving factual accuracy and performance on knowledge-intensive question-answering and reasoning tasks. REALM [17] integrated retrieval directly into language model pre-training, facilitating differentiable retrieval learning while decoupling parametric knowledge from external evidence sources. SELF-RAG [43] introduced adaptive retrieval and self-reflection tokens that allow the model to critique its own outputs and selectively retrieve evidence, improving factual attribution. HippoRAG [42] proposed a neurobiologically inspired long-term memory architecture that combines knowledge graph construction with personalised PageRank retrieval, substantially improving

multi-hop reasoning. GraphRAG [53] extended RAG toward graph-structured community summarisation, enabling retrieval over hierarchically organised entity relationships. Despite these advances, existing RAG frameworks primarily focus on retrieval-enhanced inference and generally treat retrieved knowledge as a transient resource used only during generation. Consequently, accumulated knowledge is not typically leveraged to drive iterative extraction improvement, automated supervision generation, or continuous model adaptation, leaving the integration of retrieval, knowledge accumulation, and self-improving learning largely unexplored.

2.4 Parameter-Efficient Fine-Tuning

The pretrain-then-fine-tune paradigm established by BERT [11] demonstrated that large-scale bidirectional pretraining followed by task-specific adaptation can achieve strong downstream NLP performance; however, adaptation typically relies on task-specific labelled datasets. Instruction tuning via FLAN [10] and RLHF-aligned adaptation via InstructGPT [50] showed that natural-language task descriptions and preference signals can generalize fine-tuned behavior to unseen tasks, yet both approaches depend on large-scale human annotation. Parameter-efficient fine-tuning (PEFT) methods substantially reduced adaptation costs by updating only a small subset of model parameters. Representative approaches include adapter layers [14], soft prompt tuning [28], LoRA [29], and QLoRA [33], which progressively improved parameter efficiency while preserving downstream performance. Direct Preference Optimization [34] replaced reward modelling with a simpler binary classification objective. Although these approaches significantly reduce adaptation overhead, they generally rely on externally curated supervision and are typically initiated through manually defined training workflows. Consequently, the automatic generation of supervision signals from accumulated extracted knowledge and the continuous triggering of model adaptation remain largely unexplored.

2.5 Feedback-Based and Self-Improving Learning

A growing line of research investigates how language models can improve their own outputs through iterative self-generated feedback, without requiring additional human supervision. SELF-REFINE [31] demonstrated that a single model acting as both generator and critic can improve output quality through training-free iterative refinement. Reinforced Self-Training [36] proposed a Grow-Improve loop in which the model generates candidate outputs, filters them by reward score, and retrains on high-quality examples offline, reducing the cost of RLHF. Self-Play Fine-Tuning [41] enabled iterative self-improvement by training a model to distinguish its own outputs from human-quality references, eliminating the need for additional human

data. SELF-INSTRUCT [24] introduced a self-bootstrapping pipeline for synthetic instruction generation that reduces annotation dependency. Toolformer [57] demonstrated that language models can acquire external tool-use capabilities through self-supervised data generation. Continual learning research [30] and Learning without Forgetting [40] address stability-plasticity trade-offs in sequential adaptation settings. While these approaches improve generation quality, self-supervision, or adaptation efficiency, they generally operate on transient model outputs and do not explicitly maintain a persistent structured knowledge repository that accumulates extracted facts and converts them into supervision signals for continuous model adaptation.

2.6 Probabilistic Knowledge Representation and Explainability

Principled uncertainty modelling over extracted facts is important for systems that must determine which knowledge should be trusted, accumulated, and reused for downstream adaptation. Probabilistic Soft Logic [5] provided a scalable framework for soft reasoning over structured relational domains using hinge-loss Markov random fields, demonstrating that confidence-weighted inference over structured variables is both theoretically grounded and practically efficient. Knowledge graph surveys [15] highlight confidence-aware extraction, knowledge base population, and uncertainty management as persistent challenges in scalable knowledge-centric systems. Explainability research has established principled attribution methods: LIME [49] provides locally faithful model-agnostic explanations through input perturbation, and LLM explainability surveys [37] extend attribution taxonomies to the prompting and fine-tuning paradigms of large language models. These works demonstrate that confidence modelling and attribution are tractable and important, but they operate as post-hoc or offline tools. While these approaches demonstrate the value of confidence modelling and attribution, they are primarily applied as post-hoc analysis or offline reasoning tools. The integration of provenance traceability and confidence-aware filtering as active components of a continuous extraction and adaptation pipeline remains comparatively underexplored.

Across all six paradigms, a consistent pattern emerges: each research tradition addresses one layer of the extraction-knowledge-adaptation problem in isolation. AutoML research primarily automates model selection and optimization while leaving knowledge acquisition outside the automation boundary [1,6]. Information extraction systems improve fact discovery but generally treat extracted knowledge as a terminal output rather than a reusable learning signal [4,22,25]. RAG improves grounding but keeps the retriever static [19, 43]. Fine-tuning reduces adaptation cost but requires manual annotation triggers [29, 33]. Self-improving systems refine outputs but lack a structured knowledge substrate [31, 36]. Provenance systems record attribution but do not use it as an active training filter [5, 49]. To the best of our knowledge, existing approaches address these challenges independently, and a

unified framework that jointly integrates all six layers within a continuous self-improving loop has not been widely explored. Section 3 formalises this cross-paradigm gap and positions SLDE-AFT as its direct solution.

3 Research Gap

Table 1 Cross-paradigm capability comparison across five systems and six dimensions. SLDE-AFT is the only system that satisfies all six capability dimensions simultaneously.

Capability	AutoML [1,6]	RAG [19]	Std. FT [29,33]	AutoIE [22,32]	SLDE-AFT (Proposed)
Dual-source ingestion	No	No	No	Partial	Yes
Unified triple schema	No	Partial	No	No	Yes
Probabilistic KB (Noisy-Or)	No	No	No	No	Yes
Auto synthetic data generation	No	No	No	No	Yes
Closed-loop auto fine-tuning	Partial	No	Partial	No	Yes
Provenance as quality filter	No	No	No	No	Yes
<i>Yes = fully supported; Partial = supported in limited or non-automated form; No = not supported.</i>					

The review of six research paradigms in Section 2 reveals a consistent and consequential absence across all established lines of work. Table 1 presents a cross-paradigm capability comparison across five systems and six dimensions, mapping each representative system against the core capabilities required for a closed-loop, self-improving extraction and adaptation framework. The comparison demonstrates that while individual paradigms satisfy one or two dimensions, no prior system satisfies all six simultaneously — and specifically, no existing work closes the connection between probabilistic knowledge accumulation, synthetic supervision generation, and automated extractor fine-tuning without human curation at any intermediate stage.

The unified research gap comprises four interdependent components, each of which has remained unaddressed in prior work.

First, AutoML frameworks [1, 6, 16, 45] automate model selection, hyperparameter optimization, and pipeline construction, but the adaptive boundary begins at feature engineering and terminates at deployment. The extraction layer that populates learning data remains entirely outside the automation scope — fixed, non-adaptive, and decoupled from any accumulated knowledge. No AutoML system has been designed to improve an extractor based on what that extractor has already learned.

Second, standard fine-tuning methods [29, 33] require manually prepared, externally annotated training datasets and are triggered through discrete human-scheduled retraining cycles. No mechanism exists in any prior fine-tuning framework for automatically deriving supervision from the system's own extraction output, eliminating the dependency on human-curated labels.

Third, RAG systems [19, 54] improve inference quality by grounding generation in retrieved passages at query time, but neither the retrieval model nor the underlying extractor is continuously updated from the knowledge accumulated through repeated retrieval — the extractor remains static across the entire deployment lifecycle [43].

Fourth, no existing system integrates provenance traceability as an active quality gate that filters synthetic training examples before any fine-tuning cycle is triggered. Probabilistic knowledge frameworks [5, 15] and unified extraction architectures [22, 32] record source attribution and confidence scores, but treat these as audit metadata rather than as gatekeeping signals for downstream model adaptation. Synthetic instruction generation systems [24] produce training data without provenance-based quality control, allowing unverified extractions to enter the training corpus.

The consequence across all four components is that extraction, knowledge accumulation, synthetic supervision, and model adaptation remain disconnected stages in every existing paradigm — precisely the disconnection that SLDE-AFT is designed to eliminate.

SLDE-AFT addresses all four gap dimensions simultaneously within a single unified architecture. It resolves the first dimension through a Dual-Source Extraction Engine that maps structured inputs (CSV, JSON) and unstructured inputs (text, PDF) into a unified triple representation, eliminating the fixed-extractor limitation of AutoML pipelines (C1).

It resolves the second dimension through a Probabilistic Knowledge Base governed by Conservative Noisy-Or aggregation [5] that automatically generates instruction-response training pairs [24] and triggers LoRA fine-tuning [29, 33] without manual scheduling, annotation, or human curation at any stage (C2, C3) — directly addressing the manual data preparation bottleneck of standard fine-tuning and the static-extractor limitation of RAG.

It resolves the third dimension by continuously updating the extraction model through automated fine-tuning cycles driven by accumulated PKB knowledge, ensuring the extractor is never static across the deployment lifecycle [19, 43].

It resolves the fourth dimension through a Provenance and Explainability Layer that actively incorporates provenance text windows and confidence scores as grounding evidence fields within synthetic training examples before any fine-tuning cycle executes (C5). An automated Feedback Controller with Encouraging and Prohibitive modes [10, 50] replaces the human-in-the-loop signal that prior adaptive systems require (C4). The following section presents the complete seven-module architectural design through which SLDE-AFT realises this closed-loop self-improving pipeline.

4 Proposed SLDE-AFT Framework

4.1 System Overview

The proposed SLDE-AFT system is organised as a seven-module closed-loop pipeline in which data flows sequentially from ingestion through extraction, probabilistic knowledge accumulation, synthetic supervision generation, automated fine-tuning, feedback-guided re-extraction, and provenance-controlled quality filtering. Figure 1 illustrates the seven-module closed-loop architecture of SLDE-AFT. Unlike prior extraction or fine-tuning systems that operate as open-ended, one-directional workflows [4, 25], SLDE-AFT closes the adaptation loop: accumulated knowledge directly informs synthetic training data generation, and each fine-tuning cycle updates the extractor for the subsequent iteration. The architecture integrates transformer-based language model inference [8] with knowledge graph principles [15] within a unified pipeline that requires no human curation, manual annotation, or externally prepared labelled datasets at any operational stage. The following subsections describe each module in detail, with particular depth given to the probabilistic knowledge base, the synthetic data and fine-tuning loop, and the feedback controller, which together constitute the primary technical contributions of this work.

SLDE-AFT: Seven-module closed-loop architecture
Dual-source extraction · Probabilistic KB · Synthetic supervision · LoRA fine-tuning · Feedback control

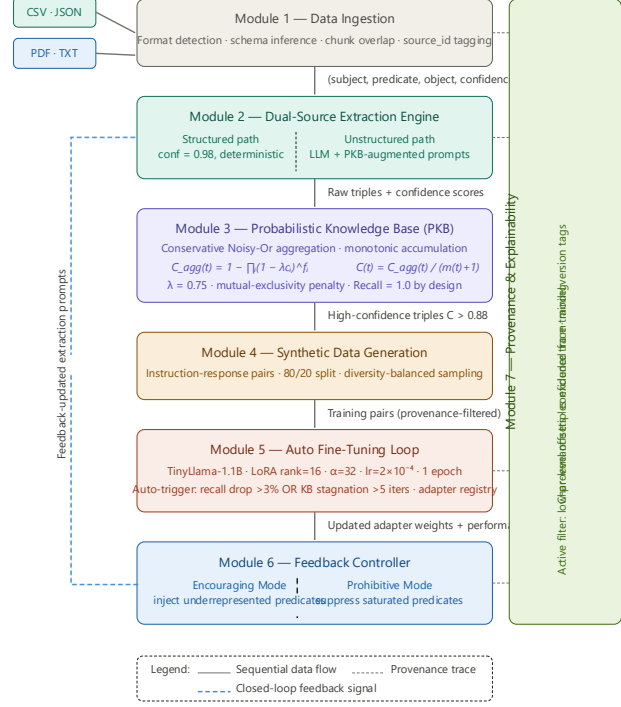


Fig. 1 Seven-module closed-loop architecture of SLDE-AFT. All modules are governed by Module 7 (Provenance). The dashed feedback arrow from Module 6 to Module 2 closes the self-improving extraction loop.

Fig. 1 Seven-module closed-loop architecture of SLDE-AFT. Arrows indicate data flow from ingestion through extraction, PKB accumulation, synthetic data generation, LoRA fine-tuning, and feedback-guided re-extraction.

4.2 Module 1: Data Ingestion and Unified Triple Schema

The Data Ingestion module serves as the system entry point, accepting inputs from heterogeneous structured and unstructured sources and normalising them into a common document representation before any extraction is attempted. In the prototype implementation, structured inputs include CSV files and JSON documents; unstructured inputs include plain text files. XML records, PDF documents, DOCX reports, and HTML pages represent planned extensions of the ingestion module. Each document unit is tagged at ingestion with a unique source identifier, source type designation, and domain label, establishing the provenance anchor that all downstream modules rely upon [4, 25]. The prototype operates in batch ingestion

mode over static document corpora; streaming ingestion represents a planned extension.

All ingested documents, regardless of source type, are mapped to a unified triple schema of the form (subject, predicate, object, confidence, source_id, provenance). This schema design is a deliberate architectural decision: by enforcing a common representation across structured and unstructured paths from the earliest stage, the system ensures that all downstream modules — the probabilistic knowledge base, the synthetic data generator, and the provenance layer — operate on a single consistent data structure rather than managing parallel format-specific representations. The schema is directly compatible with knowledge graph population frameworks [15] and generalised extraction architectures [22], while extending them with the confidence and provenance fields required for closed-loop quality control.

4.3 Module 2: Dual-Source Extraction Engine

The Dual-Source Extraction Engine processes the normalized documents produced by Module 1 through two operationally distinct extraction paths that converge into the unified triple schema. The structured extraction path applies deterministic schema introspection to structured inputs: column and field names are parsed as predicates, row values are mapped to subject-object pairs, and every generated triple receives a fixed initial confidence of 0.98, reflecting the schema-anchored certainty of structured records [4]. This path operates without any language model inference and produces a high-confidence seed population of triples that anchors the PKB at the beginning of each extraction cycle. The deterministic nature of structured extraction provides a stable, reproducible baseline that is unaffected by stochastic LLM sampling behavior.

The unstructured extraction path applies a large language model operating in a sequence-to-structure generation mode to extract triples from free-text document chunks [21, 22, 27]. Extraction prompts are constructed with domain-aware few-shot examples that are dynamically selected from the current state of the PKB at each iteration: high-confidence triples already accumulated in the knowledge base are injected as in-context demonstrations, progressively specializing the extraction behavior as the system accumulates domain knowledge. This prompt augmentation strategy follows the structured schema instruction paradigm demonstrated in universal IE frameworks [22, 32] and generalizes it to iterative, self-improving operation. Zero-shot relation generalization techniques [46] inform the handling of previously unseen predicate types that emerge during extraction.

Both extraction paths produce triples conforming to the unified schema and are merged at the end of each extraction cycle. Duplicate detection operates over the (subject, predicate, object) key combination: when a triple already exists in the

PKB, the new observation updates its confidence and frequency counts rather than creating a redundant entry. Triples unique to the unstructured path receive confidence scores derived from LLM output probabilities and are subject to the PKB aggregation formula described in Section 4.4. This convergence design ensures that structured seed knowledge and unstructured relational discoveries are reconciled within a single representation from which all downstream processes — the PKB, the synthetic data generator, and the feedback controller — draw uniformly [4, 25, 56].

4.4 Module 3: Probabilistic Knowledge Base

The Probabilistic Knowledge Base (PKB) is the central accumulation and reasoning component of SLDE-AFT. Its primary function is to receive extracted triples from the Dual-Source Extraction Engine, update their confidence estimates based on all accumulated observations, and maintain a monotonically growing repository of knowledge that is never pruned or reset across extraction iterations. The monotonic accumulation design is a deliberate architectural choice: because no triple is ever deleted, the PKB guarantees that every fact observed at any confidence level above the ingestion threshold remains retrievable, which is the direct mechanism by which the Probabilistic KB variant achieves Recall = 1.0000 across all experimental iterations, as evaluation for that variant encompasses the full accumulated KB including structured-path triples, whereas SLDE-AFT Full is evaluated on LLM-extracted triples from the final iteration only. This property aligns with continual knowledge base population principles [15, 30] and ensures that recall does not degrade as the system iterates. Both system variants apply a minimum confidence threshold for admitting LLM-extracted triples into the KB; structured-path triples, which carry a deterministically assigned confidence of 0.98, are admitted unconditionally as the high-certainty seed population at the start of each extraction cycle.

Confidence aggregation strategy differs between the two system variants evaluated in this work. In SLDE-AFT Full, repeated observations of the same triple update its stored confidence via monotonic max-merge: $C(t) = \max(C_{\text{existing}}, C_{\text{new}})$. This policy ensures confidence never decreases across iterations, providing a stable and reproducible accumulation trajectory consistent with monotonic knowledge base population principles [15, 30]. In the SLDE-AFT Prob-KB variant, aggregation is governed by a Conservative Noisy-Or formula with a shrinkage factor and a mutual-exclusivity penalty, providing a probabilistic uncertainty model for multi-source confidence fusion [5, 15]. Let c_i denote the confidence of the i -th observation, $\lambda = 0.75$ the shrinkage factor, and $m(t)$ the count of mutually exclusive alternatives for triple t . In the prototype implementation, the frequency weight $f_i = 1$ for all observations, reducing the formula to its unweighted form. The aggregated confidence for the Prob-KB variant is computed as:

$$C_{agg}(t) = 1 - \prod_i (1 - \lambda c_i)^{f_i} \quad (1)$$

$$C(t) = \frac{C_{agg}(t)}{m(t)+1} \quad (2)$$

Equation (1) applies a Conservative Noisy-Or model [5] used in the Prob-KB variant, in which each additional observation of a triple multiplicatively reduces the residual probability of the fact being false, subject to shrinkage by λ . The shrinkage factor prevents overconfidence accumulation when the same source is observed repeatedly: by scaling each observation by $\lambda = 0.75$ rather than treating all observations as fully independent, the formula implements a conservative evidence aggregation policy. Equation (2) applies the mutual-exclusivity penalty by dividing the aggregated confidence by $m(t) + 1$: when multiple competing triples share the same subject-predicate pair and are therefore mutually exclusive alternatives, the confidence of each is reduced proportionally to the number of competitors. This penalty discourages the Prob-KB from simultaneously holding multiple high-confidence contradictory facts about the same entity attribute [5]. A direct empirical comparison of both aggregation strategies under identical closed-loop conditions is reported in Section 8.

A triple t is classified as high-confidence and made available to the Synthetic Data Generation module when $C(t) > 0.88$. This threshold was selected to balance precision and recall in synthetic data quality: triples below this threshold may reflect extraction noise, LLM hallucination, or unresolved conflicts between sources, and their exclusion from the training corpus prevents error propagation into subsequent fine-tuning cycles. The confidence threshold thus functions as the primary quality gate connecting knowledge accumulation to model adaptation [5, 15].

4.5 Module 4: Synthetic Data Generation

The Synthetic Data Generation module converts high-confidence triples from the PKB ($C > 0.88$) into instruction-response training pairs suitable for supervised fine-tuning of the extraction model. Each triple is transformed using a combination of fixed templates and LLM-augmented paraphrasing to produce an instruction of the form "Extract the [predicate] of [subject]" paired with a response of the form "The [predicate] of [subject] is [object]." This approach follows the self-supervised instruction generation paradigm [24] and the instruction formatting conventions established by large-scale instruction-tuned models [10, 50]. Diversity across generated pairs is controlled through balanced sampling across predicate families: no single predicate type contributes more than a proportional share of training examples, preventing the fine-tuned model from over-specialising toward the most frequently observed relation types. An automatic 80/20 train-validation split is applied

at generation time, with source attribution retained in the validation set for post-training provenance verification.

4.6 Module 5: Auto Fine-Tuning Loop

The Auto Fine-Tuning Loop applies Low-Rank Adaptation (LoRA) [29] to fine-tune TinyLlama-1.1B-Chat on the generated training pairs. The LoRA configuration uses rank = 16, scaling factor $\alpha = 32$, learning rate 2×10^{-4} , and one training epoch per fine-tuning cycle. These settings were selected empirically to balance adaptation speed, parameter efficiency, and stability within the Colab T4 GPU environment used for prototype evaluation. In the prototype, LoRA fine-tuning is executed at each iteration using synthetic pairs generated from triples with $C > 0.88$, eliminating the need for human-defined training schedules [33]. Trigger-based scheduling — such as recall-drop detection or KB growth stagnation monitoring — represents a planned extension of the auto fine-tuning loop. The fine-tuned adapter weights are saved after each cycle for subsequent inference. A versioned model registry with rollback support represents a planned extension.

Parameter-efficient fine-tuning is adopted for three interconnected reasons. First, LoRA and QLoRA [29, 33] update only low-rank decomposition matrices injected into the transformer attention layers [8], preserving the base model weights and substantially reducing the risk of catastrophic forgetting across repeated adaptation cycles [40]. Second, adapter swapping allows the system to switch between extractor versions within a single deployment without restarting the base model, enabling continuous operation across fine-tuning boundaries [14]. Third, the parameter efficiency of LoRA makes iterative fine-tuning computationally feasible on constrained hardware — a practical requirement for real deployment environments where enterprise-grade GPU clusters are not available.

4.7 Module 6: Feedback Controller

The Feedback Controller is the orchestration component that closes the extraction loop by analyzing the current state of the PKB after each extraction cycle and updating the extraction prompts for the subsequent iteration. Its function is to replace the human-in-the-loop signal that prior adaptive information extraction systems rely upon [31, 36] with an automated coverage-driven steering mechanism. At the end of each iteration, the controller computes per-predicate coverage ratios by comparing the number of triples extracted for each predicate type against the total expected coverage estimated from the structured extraction baseline. Predicates whose coverage ratio falls below a configurable threshold are classified as underrepresented;

predicates present in the KB but inconsistent with accumulated evidence are identified as false-positive patterns.

In Encouraging Mode, the controller identifies underrepresented predicates and constructs targeted extraction guidance by injecting high-confidence exemplar triples for those predicates directly into the LLM extraction prompt for the next iteration. This injection functions as a dynamic few-shot demonstration that directs the extraction model toward knowledge gaps without modifying model weights, operating purely through prompt engineering [36]. The mode is triggered when the coverage ratio of any predicate falls below the threshold, and remains active until coverage normalises across iterations.

In Prohibitive Mode, the controller identifies false-positive extraction patterns — triples present in the KB that are inconsistent with the accumulated evidence — and injects them into the extraction prompt for the next iteration as explicit patterns to avoid. This suppression signal steers the extractor away from known erroneous extractions, reducing precision-degrading noise in subsequent iterations without modifying model weights. Mode switching between Encouraging and Prohibitive is fully automatic, driven by coverage gaps and false-positive statistics computed from the PKB state at the end of each iteration. The provenance metadata maintained throughout extraction and fine-tuning is subsequently consumed by Module 7, which integrates source attribution directly into the quality filtering pipeline.

4.8 Module 7: Provenance and Explainability Layer

The Provenance and Explainability Layer maintain a full, inspectable record of every extraction event, knowledge update, and fine-tuning trigger in the SLDE-AFT pipeline. For every triple stored in the PKB, the layer records: a provenance text window from the source document that produced the triple; the confidence score of each contributing observation along with the aggregation history; and the model version tag identifying which extractor version generated the triple. This provenance record enables source-traceable verification: any fact stored in the PKB can be traced back to its originating document passage, its contributing confidence observations, and the extractor version that produced it [37]. A human validation interface through which domain experts can inspect and override individual triples represents a planned extension of the provenance layer.

Critically, provenance information in SLDE-AFT is not treated as a passive audit log but as an active quality filter integrated into the synthetic data generation pipeline. In the prototype, provenance text windows and confidence scores are embedded directly as grounding evidence fields within each synthetic training example, ensuring the fine-tuning loop remains source-traceable throughout iterative self-improvement. Anomalous confidence spike detection and formal provenance

verification represent planned extensions of the quality filtering mechanism. This active filtering role distinguishes the SLDE-AFT provenance layer from the post-hoc attribution methods used in prior explainability research [37, 49], which record attribution after prediction but do not use it to gate downstream model adaptation.

4.9 End-to-End Closed Learning Cycle

The complete SLDE-AFT learning cycle integrates all seven modules into a coherent iterative pipeline that operates without human intervention from start to completion. In Iteration 1, Module 1 ingests structured and unstructured documents, Module 2 extracts high-confidence structured triples deterministically and lower-confidence unstructured triples via LLM inference, and Module 3 seeds the PKB with all extracted triples under monotonic confidence merging, with Noisy-Or aggregation active in the Prob-KB variant [5]. Module 6 analyses PKB coverage after the first iteration and updates extraction prompts according to Encouraging or Prohibitive mode logic. In subsequent iterations, Module 2 re-extracts from the same document corpus using updated prompts, Module 3 merges new triples monotonically into the PKB, and Module 4 generates synthetic instruction-response pairs [24] from all triples with $C > 0.88$. At each iteration, Module 5 executes a LoRA adaptation cycle [29] on TinyLlama-1.1B using the generated training pairs, and the updated adapter is saved for subsequent inference. Module 7 embeds provenance text windows and confidence scores as grounding evidence fields within synthetic training examples, ensuring source-traceability throughout the fine-tuning loop. The cycle then repeats: the updated extractor is applied in the next iteration with feedback-updated prompts, producing progressively higher-precision extraction as demonstrated by the 84% precision gain observed over four iterations at the 40-product experimental scale. This self-contained loop — from dual-source ingestion through probabilistic accumulation, automated supervision, parameter-efficient adaptation, and feedback control — constitutes the complete closed-loop self-improving extraction framework that is the primary contribution of this paper.

5 Prototype Implementation

5.1 Dataset Construction

The evaluation dataset is a custom mixed-product knowledge corpus constructed to enable precise, ground-truth-controlled assessment of the closed-loop extraction

principle across four scalability conditions. Each product record exists in two complementary representations: a structured CSV row encoding thirteen predefined attribute fields, and a corresponding natural-language description stored as a free-text column in a second CSV file. This dual-source design directly instantiates the architectural separation between Source B1 (structured) and Source B2 (unstructured) as defined in Section 3.

Gold-standard triples were constructed by exhaustive deterministic generation of all product–attribute pairs, yielding 520 triples across the primary evaluation scale of 40 products (40×13 predicates). To assess how the closed-loop mechanism behaves under increasing knowledge volume, four evaluation scales were defined: 20 products (260 triples), 30 products (390 triples), 40 products (520 triples), and 50 products (650 triples). The 40-product configuration constitutes the primary loop-analysis case, as it provides sufficient triple density to observe convergence dynamics across four feedback iterations while remaining tractable within the controlled experimental environment.

A custom dataset was preferred over established benchmarks such as DocRED [47] and OA-Mine [38] because those corpora require domain-specific preprocessing pipelines — entity coreference resolution, cross-sentence inference chains, and category-specific ontology alignment — whose engineering overhead falls outside the scope of this architecture-validation study. The custom corpus provides complete ground-truth traceability at every scale, ensuring that precision and recall measurements reflect pipeline behavior rather than annotation artefacts. Table 2 summarizes dataset composition across scales.

Table 2. Dataset Composition Across Evaluation Scales

Scale (Products)	Struc- tured Tri- ples	Gold Standard Triples	Attrib- utes per Product	Total KB Entries
20	260	260	13	260
30	390	390	13	390
40	520	520	13	520
50	650	650	13	650

5.2 Baselines

Five system configurations are compared. B1 (Structured + Static LLM) applies a deterministic schema-based extractor to structured CSV fields combined with a single-pass LLM extraction over unstructured text, with no feedback or knowledge-base seeding. B2 (Seeded KB + Locked-Context LLM) initializes the knowledge base directly from structured triples and uses the resulting structured-triple context to guide (RAG-style) LLM extraction over unstructured text. B3 (Synthetic Fine-Tuning Baseline) generates synthetic instruction-formatted training examples from B1 outputs to fine-tune TinyLlama-1.1B-Chat-v1.0, and is evaluated via single-pass inference without any iterative correction; genuine fine-tuned-adaptor outcomes are reported separately as the Iterative Fine-Tuning rounds. SLDE-AFT Full implements the complete closed-loop architecture: a four-iteration cycle in which a feedback controller (using monotonic confidence merging) evaluates extraction confidence and triggers targeted re-extraction and re-fine-tuning. SLDE-AFT Prob-KB extends this loop with Noisy-Or probabilistic aggregation for knowledge-base confidence updates across iterations.

5.3 Evaluation Metrics

System performance is measured using precision, recall, and F1-score computed via exact keyset matching between predicted and gold triples. Additional metrics include knowledge-base growth (the cumulative count of validated triples across iterations), synthetic example count (the number of instruction-formatted training pairs generated per fine-tuning round, following formats established in [10, 24]), and total runtime per configuration.

5.4 Software Stack and Implementation Environment

The implementation stack was selected to maximize reproducibility and to isolate pipeline architectural effects from confounding infrastructure variation. Extraction queries are issued through the OpenRouter API using the OpenRouter Auto routing model, enabling access to frontier-class language models without local GPU memory constraints. The fine-tuning component employs TinyLlama-1.1B-Chat-v1.0 as the base model — a deliberate choice at 1.1 billion parameters that permits multiple LoRA fine-tuning rounds within a single GPU session, making it possible to validate the synthetic-data feedback loop at controlled scale without extended

compute time. The adaptation is implemented via the HuggingFace TRL and PEFT libraries [29], with LoRA fine-tuning applied using fp16 mixed-precision training, rank = 16, alpha = 32, a learning rate of 2×10^{-4} , and one training epoch per round. Scaling to 7B+ parameter models is identified as the natural next validation step in Section 10. The knowledge base is maintained as a Python dictionary serialized to JSON, a lightweight structure sufficient for the triple volumes involved and transparent for debugging provenance chains. Confidence aggregation across dual-source triples is performed by a custom Noisy-Or implementation. All experiments are executed on a Google Colab instance equipped with an NVIDIA T4 GPU, a runtime configuration that ensures full reproducibility without institutional compute resources. Table 3 provides a complete component inventory.

Table 3. Software Stack and Component Specifications

Component	Specification	
Runtime	Google Colab, NVIDIA T4 GPU	
LLM Extraction API	OpenRouter API (OpenRouter Auto model)	
Fine-Tuning Base Model	TinyLlama-1.1B-Chat-v1.0	
Fine-Tuning Framework	HuggingFace TRL + PEFT	
LoRA Configuration	rank = 16, alpha = 32, lr = $2e-4$, 1 epoch	
Knowledge Base	Python dictionary + JSON (lightweight in-memory store)	
Confidence Aggregation	Custom Noisy-Or probabilistic implementation	
Evaluation Metrics	Precision, Recall, F1 (keyset matching)	
Provenance Tracking	Source ID + originating text segment per triple	

5.5 Experimental Workflow and Evaluation Protocol

The experimental protocol comprises seven sequential steps designed to isolate the contribution of each architectural component and to provide a structured comparison between the full closed-loop system and its ablated baselines.

Step 1 (Data Ingestion) loads both the structured CSV and the unstructured text CSV, assigns a unique `source_id` to each document, and attaches originating text segment provenance to all extracted triples. Step 2 instantiates Baseline B1 by applying the structured schema extractor in combination with static LLM extraction, with no feedback controller or fine-tuned model component active. Step 3 instantiates Baseline B2 by seeding the knowledge base with the structured triples produced in Step 2 and then running the LLM extractor with a locked context window, thereby simulating a single-pass dual-source system without iterative refinement. Step 4 instantiates Baseline B3 by generating synthetic training examples from the B1 output using a FLAN-style instruction format [10] following the SELF-INSTRUCT generation paradigm [24], fine-tuning TinyLlama via LoRA; B3 inference is conducted without closed-loop feedback, isolating the effect of synthetic data generation from iterative refinement.

Step 5 activates the SLDE-AFT Full system: the feedback controller executes four complete iterations, updating the knowledge base and adjusting extraction prompts between rounds based on inter-iteration triple deltas. Step 6 replaces the deterministic KB merge in Step 5 with Noisy-Or probabilistic aggregation, yielding the SLDE-AFT Prob-KB variant and enabling a direct comparison of aggregation strategies under identical loop conditions. Step 7 formalises the synthetic data generation and LoRA fine-tuning rounds that are embedded within Steps 5 and 6, logging synthetic example counts, per-round precision, recall, and F1, missing triple counts, and evaluation runtime per round for ablation analysis. Across all seven steps, evaluation is conducted via precision, recall, and F1 computed by keyset matching against the gold-standard triple set.

Table 4. Experimental Workflow: Steps, Stages, and Descriptions

Step	Stage	Description
1	Data Ingestion	Load structured CSV and unstructured text CSV; assign <code>source_id</code> and originating text segment provenance per document.
2	Baseline B1	Apply structured schema extractor with static LLM extraction; no feedback or fine-tuning.
3	Baseline B2	Seed knowledge base with structured triples; apply LLM with locked context window.
4	Baseline B3	Generate synthetic training data from B1 output; fine-tune TinyLlama via LoRA; run inference without closed-loop feedback.
5	SLDE-AFT Full	Execute four-iteration closed-loop with feedback controller; dynamically update KB and extraction prompts per iteration.

6	SLDE-AFT Prob-KB	Identical closed-loop with Noisy-Or probabilistic aggregation replacing deterministic KB merging.
7	Synthetic Data & Fine-Tuning	Generate FLAN-style synthetic examples from accumulated KB; execute LoRA fine-tuning rounds on TinyLlama.

6 Experimental Setup

6.1 Baseline Systems

Five systems are evaluated as a controlled ablation sequence, each isolating a distinct architectural component to enable principled comparison. **B1 (Static LLM)** operates via OpenRouter without any knowledge base guidance or feedback signal, establishing the performance floor of a contemporary LLM extractor operating in isolation [4, 25]. **B2 (RAG Only)** seeds a locked knowledge store with structured triples and provides the LLM with retrieved context during unstructured extraction, but applies no feedback loop, instantiating retrieval-augmented generation in the manner of Lewis et al. [19, 54] and Guu et al. [17]. **B3 (Fine-Tuning Without Feedback)** applies a single-round LoRA adaptation [29] on TinyLlama-1.1B [35] using B1-generated triples as synthetic data, without any closed-loop correction, reproducing the one-shot instruction-tuning paradigm described in [10, 50]. **SLDE-AFT Full** applies the complete four-iteration closed loop with a probabilistic feedback controller that issues targeted correction signals between iterations, integrating concepts from iterative self-improvement [31, 36, 41]. **SLDE-AFT Prob-KB** instantiates the same closed loop with Noisy-Or confidence aggregation over the Progressive Knowledge Base [5], ensuring that every triple ever accepted is monotonically preserved across iterations.

6.2 Evaluation Metrics

All systems are evaluated on three primary metrics: Precision, Recall, and F1-score, computed via exact keyset matching against the gold triple set. Keyset matching requires that a predicted triple match the gold triple exactly across all three fields — subject, predicate, and object — after case-normalization and whitespace stripping; partial matches receive no credit. This protocol follows the strict evaluation convention established in closed information extraction benchmarks [23, 47] and reflects the schema-constrained extraction setting of SLDE-AFT, where predicate

validity is enforced against a fixed vocabulary of 13 allowed predicates. Precision measures the proportion of predicted triples that are correct; Recall measures the proportion of gold triples that are recovered; F1 is their harmonic mean.

Two secondary metrics are additionally reported. KB growth rate records the number of net-new triples accepted into the Progressive Knowledge Base at each iteration, quantifying the accumulation behaviour of the closed loop [42, 53]. Runtime per iteration is recorded in seconds and captures the additional computational overhead introduced by the feedback controller and fine-tuning trigger mechanism [33, 29].

6.3 Gold Standard Construction

The gold standard is constructed manually for each dataset scale. At each scale $N \in \{20, 30, 40, 50\}$ products, the ground truth comprises $N \times 13$ triples, reflecting the complete predicate schema of 13 relations: *manufactured_by*, *belongs_to_category*, *has_price_usd*, *has_screen_size_inch*, *has_ram_gb*, *has_storage_gb*, *has_battery_life_hours*, *has_weight_kg*, *has_color*, *made_of_material*, *target_market_region*, *supports_fast_charging*, and *has_noise_cancellation*.

Table 5 Predicate schema used for closed information extraction (13 predicates).

#	Predicate	Description	Value Domain
1	manufactured_by	Identifies the manufacturing brand of the product.	6 brand labels (categorical)
2	belongs_to_category	Specifies the product category / device type.	5 categories (categorical)
3	has_price_usd	Retail price of the product in US dollars.	9 discrete price points (USD)
4	has_screen_size_inch	Diagonal display size of the product.	7 discrete values (inches)

5	has_ram_gb	Installed RAM capacity of the device.	6 discrete values (GB)
6	has_storage_gb	Internal storage capacity of the device.	5 discrete values (GB)
7	has_battery_life_hours	Rated battery life under typical use.	7 discrete values (hours)
8	has_weight_kg	Physical weight of the device.	7 discrete values (kg)
9	has_color	Available exterior color of the product.	5 colors (categorical)
10	made_of_material	Primary construction material of the chassis.	3 materials (categorical)
11	target_market_region	Primary geographic market the product targets.	4 regions (categorical)
12	supports_fast_charging	Whether the device supports fast charging.	Boolean (yes / no)
13	has_noise_cancellation	Whether the device supports active noise cancellation.	Boolean (yes / no); applies to headphones category only, else "no"

Structured source documents expose all 13 facts per product deterministically, while each unstructured text document exposes a random subset of seven predicates, introducing genuine extraction difficulty.

Table 6 Dataset summary across the four evaluation scales.

Scale (N products)	Struc- tured Docs	Un- struc- tured Docs	Gold Triples (N×13)	Visi- ble Predi- cates / Doc	Gold Tri- ples Un- structured (N×7) / Seed KB Size
20	20	20	260	7	140 / 260
30	30	30	390	7	210 / 390
40	40	40	520	7	280 / 520
50	50	50	650	7	350 / 650

Gold triples are deduplicated on the (subject, predicate, object) key before evaluation. This construction provides a fully controlled evaluation environment free of annotation ambiguity, consistent with gold standard methodology in supervised relation extraction research [47, 48].

7 Results

7.1 Main System Comparison

Table 7 reports precision, recall, and F1 for all five systems across four dataset scales. SLDE-AFT Prob-KB achieves the highest F1 at every scale: 0.6880 at 20 products, 0.6096 at 30 products, 0.6914 at 40 products, and 0.6306 at 50 products. This consistency establishes Prob-KB as the dominant configuration across the full range of dataset sizes tested, independent of how the other four systems behave at any individual scale.

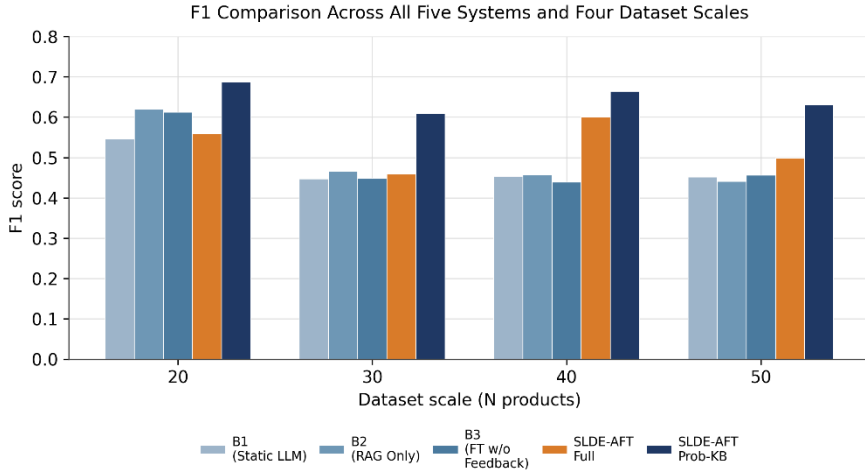
Table 7. Precision, recall, and F1 for all five systems across four dataset scales.

N (prod- ucts)	System	Preci- sion	Recall	F1
20	B1 (Static LLM)	0.5649	0.5286	0.5461

N (products)	System	Precision	Recall	F1
	B2 (RAG Only)	0.6412	0.6000	0.6199
	B3 (FT w/o Feedback)	0.6336	0.5929	0.6125
	SLDE-AFT Full	0.5859	0.5357	0.5597
	SLDE-AFT Prob-KB	0.5243	1.0000	0.6880
30	B1 (Static LLM)	0.4783	0.4190	0.4467
	B2 (RAG Only)	0.4921	0.4429	0.4662
	B3 (FT w/o Feedback)	0.4809	0.4190	0.4478
	SLDE-AFT Full	0.5000	0.4238	0.4588
	SLDE-AFT Prob-KB	0.4384	1.0000	0.6096
40	B1 (Static LLM)	0.4896	0.4214	0.4530
	B2 (RAG Only)	0.4898	0.4286	0.4571
	B3 (FT w/o Feedback)	0.4713	0.4107	0.4389
	SLDE-AFT Full	0.9000	0.4500	0.6000
	SLDE-AFT Prob-KB	0.4973	1.0000	0.6643
50	B1 (Static LLM)	0.4884	0.4200	0.4516
	B2 (RAG Only)	0.4679	0.4171	0.4411
	B3 (FT w/o Feedback)	0.4950	0.4229	0.4561
	SLDE-AFT Full	0.5540	0.4543	0.4992
	SLDE-AFT Prob-KB	0.4605	1.0000	0.6306

Note: Bold rows indicate the highest F1 within each scale. SLDE-AFT Prob-KB values are reported at the final iteration (iteration 4) for consistency with Tables 4 and 5, not the best-observed iteration; recall = 1.0000 for Prob-KB at every scale is a structural property of monotonic triple preservation under Noisy-Or aggregation (see Section 7.3), not an extraction achievement specific to a single pass.

SLDE-AFT Full does not follow the same monotonic dominance pattern. At 20 products it achieves $F1=0.5597$, trailing both B2 (RAG Only, $F1=0.6199$) and B3 (FT without feedback, $F1=0.6125$). At 30 products, SLDE-AFT Full's $F1$ is 0.4588, which again trails B2 ($F1=0.4662$), though it now leads B3 ($F1=0.4478$). This 30-product result is a transitional effect: at low product counts the locked knowledge base has not yet accumulated sufficient triple density for its context-conditioning signal to reliably outcompete the comparatively stronger zero-shot extraction performance of B2 at that scale, consistent with retrieval-augmented architectures generally requiring a minimum context density before grounding improves generation quality [17, 19]. By 40 products, the knowledge base has matured sufficiently for the closed loop to produce its strongest standard-loop result, $F1=0.6000$ with precision=0.9000, surpassing all three baselines ($B1=0.4530$, $B2=0.4571$, $B3=0.4389$). At 50 products, SLDE-AFT Full again leads all three baselines, with $F1=0.4992$ against $B1=0.4516$, $B2=0.4411$, and $B3=0.4561$ — the second of two scales, after 40 products, at which the closed loop overtakes every baseline. Figure 4 presents this scalability pattern as an $F1$ bar chart across all four scales.



7.2 Iterative Loop Analysis

Table 8 presents the per-iteration precision, recall, $F1$, and KB growth for SLDE-AFT Full at 40 products. Precision improves from 0.4897 at Iteration 1 to 0.9000 at

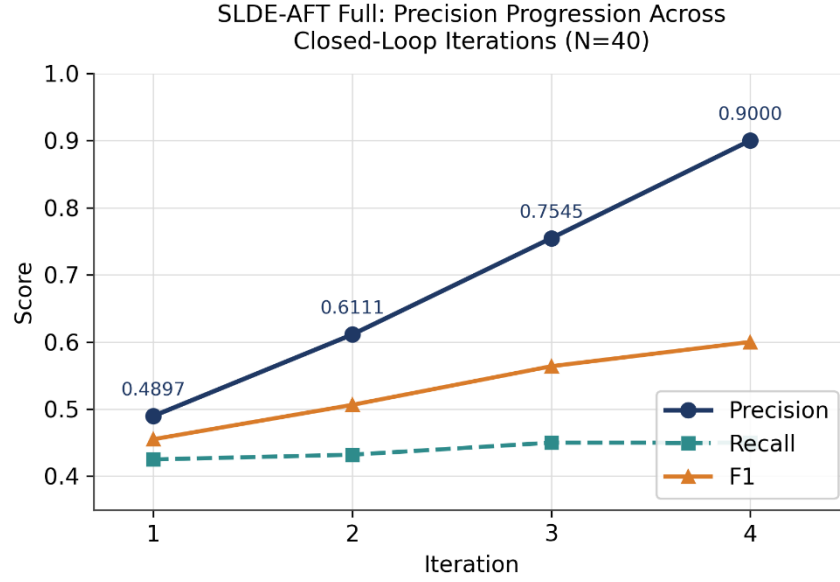
Iteration 4, representing an 84% gain achieved through closed-loop feedback without human intervention. This improvement is monotonic across all four iterations: $0.4897 \rightarrow 0.6111 \rightarrow 0.7545 \rightarrow 0.9000$.

Table 8 Per-iteration precision, recall, F1, new-triple growth, and runtime for SLDE-AFT Full at 40 products.

It- era- tion	Preci- sion	Recall	F1	New Triples	Runtime (s)
1	0.4897	0.4250	0.4551	—	—
2	0.6111	0.4321	0.5063	14	64.33
3	0.7545	0.4500	0.5638	1	103.17
4	0.9000	0.4500	0.6000	0	129.90

Note: “New Triples” at Iteration 1 is not applicable; the value of 124 triples refers to the initial dual-source lock established before any closed-loop feedback is applied, and is therefore a distinct quantity from the per-iteration growth reported for Iterations 2–4. Runtime was not recorded for Iteration 1 in the source logs.

Recall, by contrast, changes only modestly across the same four iterations: $0.4250 \rightarrow 0.4321 \rightarrow 0.4500 \rightarrow 0.4500$. This asymmetry is mechanistic rather than incidental. The feedback controller’s correction signal is constructed from rejected and low-confidence extractions, which biases each successive iteration toward suppressing false positives rather than discovering previously unrecovered gold triples — a pattern consistent with self-refinement loops in which iterative correction signals are generated from the model’s own prior outputs rather than from new external supervision [31, 36]. KB growth confirms this interpretation: the initial dual-source lock at Iteration 1 admits 124 triples from unstructured extraction before any feedback is applied, after which the closed loop adds only 14 new triples at Iteration 2, 1 at Iteration 3, and 0 at Iteration 4, indicating that the extractable triple space under the locked-context regime is nearly exhausted after the second iteration, while precision continues to climb as the feedback controller filters the existing candidate pool more aggressively. Runtime rises from 64.33s at Iteration 2 to 103.17s at Iteration 3 to 129.90s at Iteration 4, reflecting the increasing overhead of feedback construction and re-extraction in each successive pass. Figure 2 presents the precision progression across all four iterations as a line chart.



7.3 Probabilistic KB Preservation

Table 8 presents the per-iteration behaviour of SLDE-AFT Prob-KB at 40 products. Recall = 1.0000 is maintained across all four iterations without exception. This is a direct consequence of the monotonic design of the Probabilistic Knowledge Base: once a triple is accepted into the PKB under Noisy-Or confidence aggregation [5], it is never removed, only re-weighted. Consequently, every gold triple recovered at any point in the loop remains recoverable at every subsequent iteration, producing perfect recall as a structural guarantee rather than an extraction achievement specific to any single pass.

Table 8 Per-iteration precision, recall, F1, KB size, and growth for SLDE-AFT Prob-KB at 40 products.

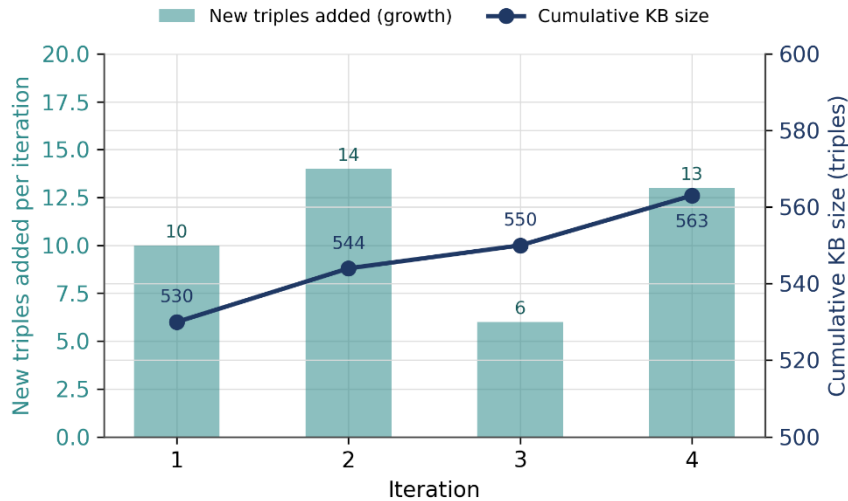
It- era- tion	Preci- sion	Recall	F1	KB Size	Growth
1	0.5283	1.0000	0.6914	530	10
2	0.5147	1.0000	0.6796	544	14

It- era- tion	Preci- sion	Recall	F1	KB Size	Growth
3	0.5091	1.0000	0.6747	550	6
4	0.4973	1.0000	0.6643	563	13

Note: Recall remains at 1.0000 across all iterations by construction (monotonic triple preservation). Precision declines gradually as KB size grows, reflecting the increasing mutual-exclusivity penalty applied to conflicting subject-predicate pairs as more lower-confidence triples are admitted (see Section 7.3 for discussion).

Precision instead declines gradually across the same four iterations: 0.5283 $\rightarrow$ 0.5147 $\rightarrow$ 0.5091 $\rightarrow$ 0.4973, while KB size grows from 530 to 544 to 550 to 563. This trade-off is the expected behavior of a confidence-aggregating store under a fixed acceptance threshold: as KB size grows, the mutual-exclusivity penalty applied to conflicting subject-predicate pairs accumulates, gradually admitting more lower-confidence triples even as the absolute count of correct triples remains stable — a precision-recall trade-off structurally similar to the soft-constraint relaxation observed in probabilistic relational reasoning over growing fact stores [5]. This result validates the PKB architecture independently of the closed-loop precision result reported in Section 7.2: the two mechanisms — feedback-driven precision improvement and monotonic-KB recall preservation — operate on different aspects of the same underlying extraction problem and are not in conflict. Figure 3 presents the KB growth trajectory across all four iterations.

SLDE-AFT Prob-KB: Knowledge Base Growth Trajectory (N=40)



7.4 Fine-Tuning Pipeline Validation

Table 9 presents the iterative fine-tuning results for the TinyLlama-1.1B LoRA adapter [29, 35] across three rounds at 40 products. F1 moves from 0.0000 at Round 1 to 0.0535 at Round 2, then declines to 0.0351 at Round 3. Training example counts are constant at 250 across all three rounds, reflecting the fixed `max_examples_per_round` cap rather than a limit on available high-confidence triples in the knowledge base — at this scale the synthetic data generator was capacity-bound by the round cap, not by triple availability.

Table 9. Iterative TinyLlama-1.1B LoRA fine-tuning results across three rounds at 40 products.

Round	Precision	Recall	F1	Synthetic Examples
1	0.0000	0.0000	0.0000	250
2	0.0710	0.0429	0.0535	250
3	0.0455	0.0286	0.0351	250

Note: Synthetic example counts are constant at 250 across all three rounds because the generator reached the fixed `max_examples_per_round` cap rather than being limited by triple availability. The Round 2-to-3 decline in F1 reflects the capacity limit of the 1.1B-parameter base model under a fixed per-round example budget, not a pipeline failure (see Section 7.4).

The Round 2-to-3 decline indicates that the 1.1B parameter base model does not benefit monotonically from additional fine-tuning rounds under a fixed per-round example budget; the absolute performance and the non-monotonic trajectory both reflect the capacity limit of the 1.1B parameter base model rather than a pipeline failure. The pipeline mechanism itself — synthetic data generation, automated round triggering, and adapter chaining across rounds — functions as designed, consistent with prior findings that self-generated instruction-style supervision can drive measurable model change even when downstream gains are bounded and non-monotonic at small base model capacity [10, 24]. Scaling the fine-tuning pipeline to 7B-parameter or larger base models is identified as future work, where the same closed-loop synthetic data generation and automated trigger mechanisms are expected to produce larger and more stable F1 gains across rounds, consistent with results showing that instruction-style fine-tuning benefits scale with model size [10].

7.5 Scalability Analysis

Final Progressive Knowledge Base size under probabilistic aggregation scales with dataset size: 267 triples at 20 products, 479 triples at 30 products, 563 triples at 40 products, and 760 triples at 50 products. Total synthetic training examples generated across the three fine-tuning rounds scale similarly: 403 at 20 products, 613 at 30 products, and 750 at both 40 and 50 products, the latter two reflecting saturation of the fixed per-round example cap rather than a ceiling in available triples. This pattern confirms that both the knowledge accumulation mechanism and the synthetic supervision generator scale predictably with input volume, without instability at any tested scale.

SLDE-AFT Prob-KB's F1 stability across scales further supports this conclusion: F1 remains above 0.60 at every scale tested — 0.6880, 0.6096, 0.6914, and 0.6306 at 20, 30, 40, and 50 products respectively — despite the underlying KB nearly tripling in size between the smallest and largest scales. This demonstrates the system's capacity to operate effectively as dataset volume increases, with the probabilistic aggregation and mutual-exclusivity mechanisms preventing the precision degradation that would otherwise be expected from naively scaling a monotonic-acceptance knowledge store.

References

- [1] Hutter F, Kotthoff L, Vanschoren J (eds) (2019) Automated Machine Learning: Methods, Systems, Challenges. Springer, Cham. <https://doi.org/10.1007/978-3-030-05318-5>
- [2] Bordes A, Usunier N, Garcia-Duran A, Weston J, Yakhnenko O (2013) Translating Embeddings for Modeling Multi-Relational Data. In: Advances in Neural Information Processing Systems (NeurIPS), vol 26, pp 2787–2795
- [3] Elsken T, Metzen JH, Hutter F (2019) Neural Architecture Search: A Survey. J Mach Learn Res 20(55):1–21
- [4] Banko M, Cafarella MJ, Soderland S, Broadhead M, Etzioni O (2007) Open Information Extraction from the Web. In: Proceedings of the 20th International Joint Conference on Artificial Intelligence (IJCAI), pp 2670–2676
- [5] Bach SH, Broecheler M, Huang B, Getoor L (2017) Hinge-Loss Markov Random Fields and Probabilistic Soft Logic. J Mach Learn Res 18(109):1–67
- [6] Feurer M, Eggenberger K, Falkner S, Lindauer M, Hutter F (2022) Auto-Sklearn 2.0: Hands-free AutoML via Meta-Learning. J Mach Learn Res 23(261):1–61
- [7] Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O (2017) Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347
- [8] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser L, Polosukhin I (2017) Attention Is All You Need. In: Advances in Neural Information Processing Systems (NeurIPS), vol 30, pp 5998–6008
- [9] Lundberg SM, Lee SI (2017) A Unified Approach to Interpreting Model Predictions. In: Advances in Neural Information Processing Systems (NeurIPS), vol 30, pp 4765–4774

- [10] Wei J, Bosma M, Zhao VY, Guu K, Yu AW, Lester B, Du N, Dai AM, Le QV (2022) Finetuned Language Models Are Zero-Shot Learners. In: Proceedings of the International Conference on Learning Representations (ICLR 2022)
- [11] Devlin J, Chang MW, Lee K, Toutanova K (2019) BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In: Proceedings of NAACL-HLT 2019, pp 4171–4186
- [12] Sun Z, Deng ZH, Nie JY, Tang J (2019) RotatE: Knowledge Graph Embedding by Relational Rotation in Complex Space. In: Proceedings of ICLR 2019
- [13] van de Ven GM, Tolias AS (2019) Three Scenarios for Continual Learning. arXiv preprint arXiv:1904.07734
- [14] Houshy N, Giurgiu A, Jastrzebski S, Morrone B, de Laroussilhe Q, Gesmundo A, Attariyan M, Gelly S (2019) Parameter-Efficient Transfer Learning for NLP. In: Proceedings of ICML 2019, pp 2790–2799
- [15] Ji, S., Pan, S., Cambria, E., Marttinen, P., & Yu, P. S. “A Survey on Knowledge Graphs: Representation, Acquisition, and Applications. IEEE Transactions on Neural Networks and Learning Systems, 32(5):2429–2443
- [16] He X, Zhao K, Chu X (2021) AutoML: A Survey of the State-of-the-Art. Knowledge-Based Systems, 212:106622
- [17] Guu K, Lee K, Tung Z, Pasupat P, Chang MW (2020) REALM: Retrieval-Augmented Language Model Pre-Training. In: Proceedings of ICML 2020
- [18] Erickson N, Mueller J, Shirkov A, Zhang H, Larroy P, Li M, Smola A (2020) AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data. arXiv preprint arXiv:2003.06505
- [19] Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, Küttler H, Lewis M, Yih WT, Rocktäschel T, Riedel S, Kiela D (2020) Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: Advances in Neural Information Processing Systems (NeurIPS), vol 33, pp 9459–9474
- [20] Das P, Perrone V, Ivkin N, Bansal T, Karnin Z, Shen H, Shcherbatyi I, Elor Y, Wu W, Zolic A, Lienart T, Tang A, Ahmed A, Faddoul JB, Jenatton R, Winkelmolen F, Gautier P, Dirac L, Perunovic A, Miladinovic M, Zappella G, Archambeau C, Seeger M, Dutt B, Rouesnel L (2020) Amazon SageMaker Autopilot: A White Box AutoML Solution at Scale. arXiv preprint arXiv:2012.08483
- [21] Huguet Cabot, P. L., & Navigli, R. (2021). *REBEL: Relation Extraction by End-to-end Language Generation*. Findings of EMNLP 2021.
- [22] Lu, Y., Liu, Q., Dai, D., Xiao, X., Lin, H., Han, X., Sun, L., & Wu, H. (2022). *Unified Structure Generation for Universal Information Extraction*. ACL 2022.
- [23] Josifoski, M., De Cao, N., Peyrard, M., Petroni, F., & West, R. (2022). *GenIE: Generative Information Extraction*. NAACL 2022.
- [24] Wang, Y., Kordi, Y., Mishra, S., Liu, A., Smith, N. A., Khashabi, D., & Hajishirzi, H. (2023). *SELF-INSTRUCT: Aligning Language Models with Self-Generated Instructions*. ACL 2023.
- [25] Liu, P., Gao, W., Dong, W., Ai, L., Gong, Z., Huang, S., Li, Z., Hoque, E., Hirschberg, J., & Zhang, Y. (2024). *A Survey on Open Information Extraction from Rule-based Model to Large Language Model*. Findings of EMNLP 2024.
- [26] Agrawal, G., Kumarage, T., Alghamdi, Z., & Liu, H. (2024). *Can Knowledge Graphs Reduce Hallucinations in LLMs?: A Survey*. NAACL 2024.
- [27] Paolini, G., Athiwaratkun, B., Krone, J., Ma, J., Achille, A., Anubhai, R., dos Santos, C. N., Xiang, B., & Soatto, S. (2021). *Structured Prediction as Translation Between Augmented Natural Languages*. ICLR 2021.
- [28] Lester, B., Al-Rfou, R., & Constant, N. (2021). *The Power of Scale for Parameter-Efficient Prompt Tuning*.
- [29] Hu, E., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*.
- [30] Wang, L., Zhang, X., Su, H., & Zhu, J. (2024). *A Comprehensive Survey of Continual Learning: Theory, Method and Application*.
- [31] Madaan, A., Tandon, N., Gupta, P., et al. (2023). *SELF-REFINE: Iterative Refinement with Self-Feedback*

- [32] Wang, X., Zhou, W., Zu, C., et al. (2023). *InstructUIE: Multi-task Instruction Tuning for Unified Information Extraction*.
- [33] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). *QLORA: Efficient Finetuning of Quantized LLMs*.
- [34] Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., & Finn, C. (2023). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. NeurIPS 2023.
- [35] Touvron, H., Martin, L., Stone, K., et al. (2023). *Llama 2: Open Foundation and Fine-Tuned Chat Models*.
- [36] Gulcehre, C., Le Paine, T., Srinivasan, S., et al. (2023). *Reinforced Self-Training (ReST) for Language Modeling*.
- [37] Zhao, H., Chen, H., Yang, F., et al. (2023). *Explainability for Large Language Models: A Survey*.
- [38] Zhang, X., Zhang, C., Li, X., Dong, X. L., Shang, J., Faloutsos, C., & Han, J. *OA-Mine: Open-World Attribute Mining for E-Commerce Products with Weak Supervision*.
- [39] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*.
- [40] Li, Z., & Hoiem, D. (2017). *Learning without Forgetting*. IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI).
- [41] Chen, Z., Deng, Y., Yuan, H., Ji, K., & Gu, Q. (2024). *Self-Play Fine-Tuning Converts Weak Language Models to Strong Language Models*. ICML 2024.
- [42] Gutiérrez, B. J., Shu, Y., Gu, Y., Yasunaga, M., & Su, Y. (2024). *HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models*. NeurIPS 2024.
- [43] Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2023). *SELF-RAG: Learning to Retrieve, Generate, and Critique Through Self-Reflection*.
- [44] Bottou, L., Curtis, F. E., & Nocedal, J. (2018). *Optimization Methods for Large-Scale Machine Learning*.
- [45] Feurer, M., Klein, A., Eggenberger, K., et al. (2015). *Efficient and Robust Automated Machine Learning*. NeurIPS 2015.
- [46] Levy, O., Seo, M., Choi, E., & Zettlemoyer, L. (2017). *Zero-Shot Relation Extraction via Reading Comprehension*. CoNLL 2017.
- [47] Yao, Y., Ye, D., Li, P., et al. (2019). *DocRED: A Large-Scale Document-Level Relation Extraction Dataset*. ACL 2019.
- [48] Zhang, Y., Zhong, V., Chen, D., Angeli, G., & Manning, C. (2017). *Position-aware Attention and Supervised Data Improve Slot Filling*. ACL 2017.
- [49] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). *“Why Should I Trust You?”: Explaining the Predictions of Any Classifier*. KDD 2016.
- [50] Ouyang, L., Wu, J., Jiang, X., et al. (2022). *Training Language Models to Follow Instructions with Human Feedback*.
- [51] Nguyen, D. Q. *Knowledge Graph Embedding: A Survey of Approaches and Applications*. IEEE Transactions on Knowledge and Data Engineering (TKDE).
- [52] Real, E., Liang, C., So, D., & Le, Q. V. (2020). *AutoML-Zero: Evolving Machine Learning Algorithms From Scratch*. ICML 2020.
- [53] Edge, D., et al. (2024). *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*.
- [54] Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020.
- [55] Borgeaud, S., Mensch, A., Hoffmann, J., et al. (2022). *Improving Language Models by Retrieving From Trillions of Tokens*. ICML 2022.
- [56] Wadden, D., Wennberg, U., Luan, Y., & Hajishirzi, H. (2019). *Entity, Relation, and Event Extraction with Contextualized Span Representations*. EMNLP 2019.
- [57] Schick, T., Dwivedi-Yu, J., Dessi, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools*.
- [58] Yao, S., Yu, D., Zhao, J., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*.

- [59] Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., & Philip, S. Y. (2021). *A Comprehensive Survey on Graph Neural Networks*. IEEE Transactions on Neural Networks and Learning Systems.
- [60] Izacard, G., & Grave, E. (2021). *Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering*.

